

CI/CD + FastAPI + Containers for Microservices

1 What is CI/CD?

CI/CD stands for:

Term	Meaning
CI → <i>Continuous Integration</i>	Automating the process of testing and merging code
CD → <i>Continuous Delivery/Deployment</i>	Automatically building, testing, and releasing code to production

Together, they make ML pipelines **automated, fast, and error-free**.

Why It Matters for MLOps

In MLOps, you don't just write code once — you:

- Keep updating data
- Retrain models
- Fix preprocessing code
- Re-deploy APIs

Manually doing this every time would take forever.
So CI/CD ensures:

- New changes are automatically tested
- Models are versioned and deployed safely

- Teams collaborate without breaking each other's code
-

2 Continuous Integration (CI)

Goal: Automate testing, linting, and validation whenever someone pushes new code.

Typical CI steps for an ML project:

1. Pull latest code from GitHub
2. Install dependencies (`requirements.txt`, `environment.yml`)
3. Run unit tests (e.g., model output shape, accuracy)
4. Run code quality checks (linting, formatting)
5. Log results (pass/fail)

If all tests pass → code merges into main branch.

Tools used:

- GitHub Actions
 - GitLab CI
 - Jenkins
 - CircleCI
 - Azure DevOps Pipelines
 - AWS CodeBuild
-

3 Continuous Delivery / Continuous Deployment (CD)

Goal: Automatically release or deploy new versions of your model or API.

Steps:

1. Build a **Docker image** of the latest code/model
2. Push it to a **container registry** (like Docker Hub, AWS ECR, or GCP Artifact Registry)
3. Deploy automatically to cloud (AWS, Azure, GCP, or Kubernetes cluster)
4. Expose endpoint via **FastAPI / Flask / REST API**

Example flow:

Code Push → CI checks → Docker Build → Deploy → FastAPI Endpoint Live

4 CI/CD in an MLOps Lifecycle

Stage	What Happens
Data Stage	Data validation scripts triggered
Training Stage	Model training pipeline runs automatically
Testing Stage	Evaluate accuracy, drift, and metrics
Packaging Stage	Build Docker container
Deployment Stage	Deploy model as REST API
Monitoring Stage	Collect logs, metrics, and trigger retraining

5 Containers and Microservices

What is a Container?

A **container** is a lightweight, portable package that contains everything needed to run your application — **code + dependencies + environment**.

Popular container tool → **Docker**.

Why use containers?

- They run *exactly the same* everywhere (no “works on my machine” issues)
- Easily scalable
- Isolated environments for each service

Example:

```
docker build -t mymodel:v1 .  
docker run -p 8000:8000 mymodel:v1
```

Microservices in ML

In large systems, we break one big app into **small, independent services**:

Service	Role
<code>data-service</code>	handles preprocessing
<code>train-service</code>	retrains model periodically
<code>serve-service</code>	hosts prediction API (FastAPI)
<code>monitor-service</code>	checks drift & sends alerts

All are deployed separately using **containers**.

Why?

- Easier debugging and scaling
 - If one service fails, others keep working
-

FastAPI for Model Serving

FastAPI is a Python web framework used to expose ML models as APIs.

Advantages:

- ✓ Extremely fast (built on ASGI)
 - ✓ Easy to use (like Flask, but faster)
 - ✓ Auto-generates documentation at [/docs](#)
 - ✓ Works well inside Docker containers
-

Example: Serving a Model with FastAPI

```
from fastapi import FastAPI
import joblib
import numpy as np

app = FastAPI()
model = joblib.load("model.pkl")

@app.get("/")
def home():
    return {"message": "Model API Running"}

@app.post("/predict")
def predict(data: dict):
    input_features = np.array(data["features"]).reshape(1, -1)
    prediction = model.predict(input_features)
    return {"prediction": int(prediction[0])}
```

Run locally:

```
uvicorn main:app --reload
```

Then access:

- <http://127.0.0.1:8000/predict>
 - API docs auto-generated at [/docs](#)
-

7 Dockerfile for FastAPI Microservice

```
# 1. Base image
FROM python:3.11-slim
```

2. Set work directory

WORKDIR /app

3. Copy project files

COPY . .

4. Install dependencies

RUN pip install -r requirements.txt

5. Expose port

EXPOSE 8000

6. Command to run API

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]

Build and run:

docker build -t fastapi-ml:v1 .

docker run -p 8000:8000 fastapi-ml:v1

✅ You've now containerized your FastAPI ML microservice!

8 Deploying to Cloud via CI/CD

Cloud	Deployment Option	Example Tool
AWS	Elastic Beanstalk, ECS, EKS, Lambda	AWS CodePipeline
Azure	App Service, AKS, Functions	Azure DevOps
GCP	Cloud Run, GKE, App Engine	Cloud Build, Cloud Deploy

Each cloud integrates with GitHub so every **git push** → triggers auto build → deploys container.

9 Monitoring and Updating

After deployment:

- Use **Prometheus / Grafana / CloudWatch** for API health & latency.
 - Use **MLflow or Vertex AI** for model versioning.
 - Automate retraining pipeline if drift or poor accuracy detected.
-

10 Real-world Flow Summary

1. Developer pushes code to GitHub
2. CI pipeline runs tests
3. CD builds Docker image
4. FastAPI container deployed to cloud
5. Monitoring detects drift → retraining triggered
6. New model deployed automatically

This is modern, production-grade **MLOps automation**.

15 MCQs (with answers + explanations)

- 1 CI/CD stands for → **Continuous Integration / Continuous Deployment**
- 2 Goal of CI → **Automatically test and merge new code**
- 3 Goal of CD → **Automatically release/deploy updates**
- 4 CI checks usually include → **Unit tests, linting, and build validation**
- 5 Docker container → **Lightweight package containing code + environment**
- 6 Command to build Docker image → `docker build -t name .`
- 7 Command to run a Docker container → `docker run -p 8000:8000 name`
- 8 FastAPI framework is built on → **ASGI (Asynchronous Server Gateway Interface)**
- 9 Default FastAPI docs endpoint → `/docs`
- 10 Port exposed in FastAPI container → **8000 (by default)**
- 11 Which file defines environment setup for Docker? → **Dockerfile**
- 12 Each microservice should → **Be independent and containerized**
- 13 Example CI/CD tool → **GitHub Actions / Jenkins / GitLab CI**
- 14 AWS CI/CD service → **CodePipeline**
- 15 In MLOps, CI/CD helps → **Automate retraining, testing, and redeployment of models**



10 Short Answer Questions (2–3 sentences each)

1 What is CI/CD?

It's an automation system that continuously tests, builds, and deploys ML models or APIs whenever code changes.

2 Why do we need CI/CD in MLOps?

Because models and data change frequently — CI/CD ensures reliable and repeatable deployment without manual errors.

3 What is a container?

A self-contained unit with code, dependencies, and runtime — runs identically on any machine.

4 What is the role of Docker in CI/CD?

Docker packages your ML code and model into a deployable image that the pipeline can automatically ship to servers.

5 Define a microservice.

A small, independent application (like prediction or data preprocessing) that communicates with others via APIs.

6 What is FastAPI used for?

To create and serve REST APIs for ML models efficiently and with auto-generated documentation.

7 Write the Dockerfile instruction to expose port 8000.

`EXPOSE 8000`

8 What triggers CI/CD pipelines?

Usually a `git push` or pull request to the repository's main branch.

9 Why use FastAPI instead of Flask?

It's faster (asynchronous), has type hints, and auto-generates Swagger documentation.

10 How do containers help scalability?

You can run multiple copies (replicas) of a service instantly across cloud servers — useful for large traffic prediction workloads.